

PROOF OF CONCEPT

AI Company Knowledge Assistant

An Enterprise RAG-Based AI Assistant for Internal Knowledge Discovery Across Wikis, Docs & Policies

Document Code: POC-KNOW-003

Classification: Internal — AI Mentor Review

Prepared by: AI Solutions Architecture Team

Version 1.0 — POC Approval Draft

Table of Contents

1. Executive Summary
2. Problem Statement
3. Proposed AI Solution
4. System Architecture
5. Complete Folder Structure
6. Functional Modules
7. APIs
8. Database Design
9. RAG Pipeline
10. LangChain + LangGraph Flow
11. Tech Stack
12. Production Features
13. Future Enhancements
14. Risks
15. Development Roadmap
16. Conclusion

1. Executive Summary

1.1 The Business Problem

Employees across large organizations routinely spend significant time searching for internal information — HR policies, engineering runbooks, product documentation, onboarding guides — scattered across wikis, shared drives, ticketing systems, and chat history. Existing internal search tools return long lists of loosely relevant documents rather than direct answers, forcing employees to read multiple pages or interrupt colleagues to get a simple answer.

1.2 Why This Problem Persists

Knowledge is fragmented across many disconnected systems (Confluence, SharePoint, Google Drive, Slack, ticketing tools), each with its own search index and access model, and is authored by many different teams with no consistent structure or up-to-date ownership, so no single system reliably reflects the current, correct answer to a given question.

1.3 Why Current Solutions Fall Short

Legacy keyword-based enterprise search surfaces documents ranked by term frequency rather than relevance to intent, frequently returning outdated or superseded pages above the current authoritative version, and provides no synthesized answer — the employee still has to read and reconcile multiple sources themselves.

1.4 Why AI + Retrieval-Augmented Generation (RAG)

RAG allows the assistant to retrieve the most current, permission-appropriate passages across all connected knowledge sources and synthesize a direct answer with citations, rather than returning a ranked document list that still requires manual reading and reconciliation.

1.5 Business Value

- Reduces time employees spend searching for internal information.
- Surfaces the current, authoritative version of policies and documentation, reducing reliance on outdated pages.
- Reduces repetitive interruptions to subject-matter experts for questions already documented.
- Improves new-hire onboarding speed through self-service, cited answers.
- Creates visibility into knowledge gaps via analytics on unanswered or low-confidence queries.

1.6 Target Users

Persona	Role	Primary Need
New Employee	Onboarding and learning internal processes	Fast answers without needing to know where to look
Engineer	Needs runbooks, architecture docs, and past decisions	Precise, cited technical answers
HR/People Team	Fields repetitive policy questions	Deflection of routine queries to self-service
People Manager	Needs org policy and process	Reliable, up-to-date answers with source

Persona	Role	Primary Need
	clarity	links

1.7 Expected Impact

- Target: significant reduction in average time-to-answer for internal knowledge questions.
- Target: measurable reduction in repetitive questions routed to HR/IT/Eng support channels.
- Faster new-hire ramp-up through self-service knowledge access.
- Improved documentation hygiene driven by visibility into low-confidence and unanswered queries.

2. Problem Statement

2.1 Detailed Problem Description

As organizations grow, internal knowledge accumulates across an increasing number of disconnected tools, each optimized for authoring rather than retrieval. Employees must know not just the answer's content but which specific system and page is likely to contain it, a discovery burden that scales poorly with organizational size and headcount turnover.

2.2 Industry Examples

Organization Type	Scenario	Consequence
Fast-growing tech company	New engineer spends a full day searching for the correct deployment runbook, which exists but is buried under three renamed and outdated copies.	Delayed onboarding and repeated interruption of senior engineers for guidance.
Large enterprise HR team	Same parental-leave policy question is asked in Slack dozens of times per quarter despite being documented.	HR team time consumed by repetitive queries instead of higher-value work.
Distributed sales organization	Sales reps use outdated pricing sheets because the current version lives in a different, less-discoverable folder.	Incorrect quotes issued to prospective customers.

2.3 Current Workflow (As-Is)

1. Employee has a question and guesses which internal system likely contains the answer.
2. Employee searches that system using keyword search, receiving a long, loosely ranked result list.
3. Employee opens multiple documents to determine which is current and authoritative.
4. If no clear answer is found, employee posts in a team chat channel or DMs a colleague.
5. A subject-matter expert manually answers, repeating an answer likely given many times before.
6. The answer is not captured back into a searchable knowledge source, so the cycle repeats.

2.4 Pain Points

- No single entry point across all internal knowledge sources.
- Outdated or superseded documents frequently rank above the current authoritative version.
- High interruption cost to subject-matter experts for already-documented information.
- No visibility into which questions are frequently asked but poorly documented.
- Knowledge shared informally in chat is lost and not fed back into searchable documentation.

2.5 Compliance, Regulatory & Documentation Challenges

While less regulation-driven than other domains, internal knowledge systems must still respect data access boundaries — HR, legal, and compensation documentation must only be surfaced to employees with appropriate access, and the system must not aggregate or expose sensitive personal or confidential information beyond a user's existing permissions.

3. Proposed AI Solution

3.1 Solution Overview

The AI Company Knowledge Assistant connects to the organization's existing knowledge repositories (wikis, document stores, ticketing systems), indexes their content into a unified, permission-aware vector store, and answers employee questions in natural language with direct citations to the source page, respecting each employee's existing access permissions.

3.2 Why Retrieval-Augmented Generation Is the Correct Approach

Internal knowledge changes constantly as policies and documentation are updated, and answers must reflect the current version and respect per-user access permissions — both requirements that only a retrieval-grounded architecture querying a live, permission-aware index can satisfy.

3.3 Why a Standard Chatbot Is Not Sufficient

A standard chatbot has no access to an organization's private internal documentation and cannot know company-specific policies, processes, or terminology, making it structurally incapable of answering the vast majority of internal knowledge questions without retrieval.

3.4 Why Vector Search Is Required

Employees phrase questions in casual, varied language ('how do I expense a conference' vs. the document title 'Travel & Expense Reimbursement Policy'), so semantic vector search is required to bridge natural phrasing to the correct source content, complemented by keyword search for exact tool or system names.

3.5 How the LLM Improves the Workflow

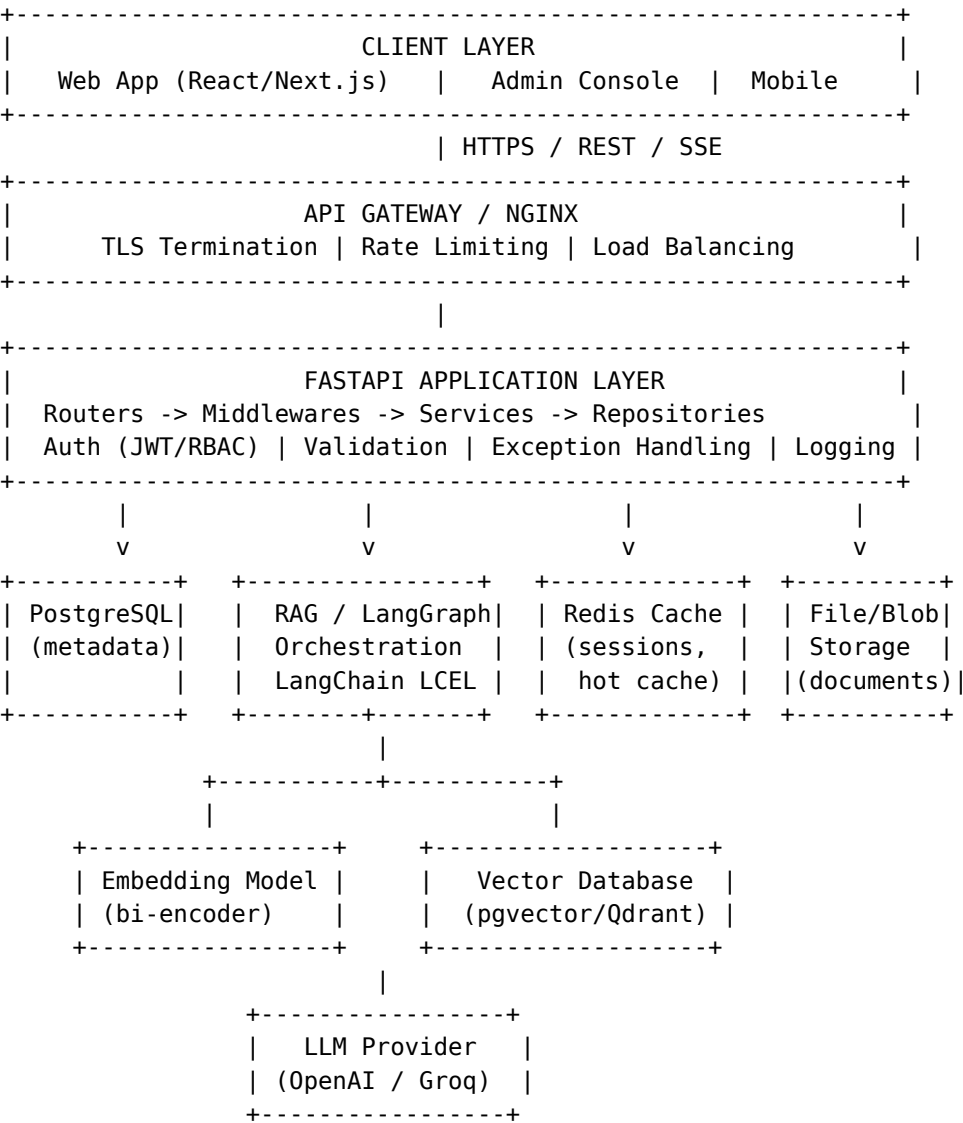
The LLM synthesizes retrieved passages — often from multiple source systems — into a single, direct, plain-language answer with citations, removing the burden of reading and reconciling several documents to answer one question.

4. System Architecture

4.1 Architectural Overview

The AI Company Knowledge Assistant platform is built as a modular, cloud-deployable, layered architecture. Each layer has a single responsibility, communicates through well-defined contracts, and can be scaled or replaced independently. This separation of concerns is essential for an enterprise system that must evolve as internal company knowledge requirements, models, and regulations change over time.

4.2 High-Level Component Diagram



4.3 Component Responsibilities

Component	Responsibility
Frontend	Chat UI, document upload, admin dashboards, citation rendering, streaming response display.
API Gateway / Nginx	TLS termination, reverse proxy, load balancing, request throttling.

Component	Responsibility
FastAPI Backend	Async REST API layer; routes requests to services; enforces auth and validation.
Authentication (JWT)	Issues and verifies signed access/refresh tokens; short-lived access, rotating refresh.
Role-Based Access Control	Maps users to roles (Admin, Reviewer, Analyst, Viewer) and enforces per-endpoint scopes.
LangChain (LCEL)	Declarative composition of prompt -> retriever -> LLM -> parser as runnables.
LangGraph	Stateful orchestration: conditional routing, retries, human-approval nodes, multi-step agents.
Document Parser	Extracts text/tables from PDF, DOCX, TXT, HTML while preserving structure and page numbers.
Chunking Engine	Splits parsed text into semantically coherent chunks with overlap and metadata tags.
Embedding Model	Converts chunks and queries into dense vectors for semantic similarity search.
Vector Database	Stores embeddings and metadata; supports ANN search and hybrid (keyword + vector) queries.
Retriever	Fetches top-k relevant chunks per query, applies metadata filters (date, doc type, role).
Reranker	Cross-encoder re-scoring of retrieved chunks to improve precision before context injection.
Prompt Template	Structured, versioned prompts with system instructions, guardrails, and citation formatting.
Output Parser	Validates and structures LLM output (JSON schema, citation extraction, refusal detection).
LLM	Generates grounded natural-language answers strictly from the retrieved context.
PostgreSQL	System of record for users, roles, documents, conversations, audit logs, analytics.
Redis	Session cache, rate-limit counters, short-term conversation memory, embedding cache.
File / Blob Storage	Durable storage of original source documents and processed artifacts.
Logging & Monitoring	Structured logs, metrics, tracing across every request for observability and audit.
CI/CD	Automated build, test, container image publishing, and deployment pipeline.

4.4 End-to-End Data Flow

The following sequence describes how a single user query flows through the system, from ingestion of source documents to the delivery of a grounded, cited answer:

- Documents are uploaded via the Document Upload API and stored in Blob Storage.
- The Document Parser extracts raw text and structural metadata (page, section, table).

9. The Chunking Engine splits content into overlapping chunks and tags each with source metadata.
10. The Embedding Model converts each chunk into a vector, which is written to the Vector Database alongside metadata.
11. A user submits a natural-language question through the Chat API.
12. The query is embedded and passed to the Retriever, which performs a hybrid (vector + keyword) search filtered by the user's role and permitted document scope.
13. The Reranker re-orders the top candidate chunks by relevance to the exact query.
14. LangChain (LCEL) assembles a Prompt Template containing the system instructions, the reranked context, and conversation memory.
15. LangGraph routes the request: if confidence is low or the query touches a sensitive workflow, it is routed to a human-approval node before finalization.
16. The LLM generates a grounded response with inline citations back to source chunks.
17. The Output Parser validates the response structure and citation integrity before it is returned to the client.
18. The interaction, retrieved sources, and response are persisted to PostgreSQL for audit and analytics; the response is streamed to the client over SSE.

5. Complete Folder Structure

5.1 Repository Layout

```

src/
  app/
    routers/          # FastAPI route definitions grouped by domain (auth, chat, documents,
admin)                # Business logic orchestrating repositories, RAG pipeline, and external
    services/          # Data access layer – encapsulates all PostgreSQL/ORM queries
calls                 # DB session management, connection pooling, Alembic migrations
    repositories/      # SQLAlchemy ORM models (User, Document, Conversation, AuditLog, etc.)
    database/          # Pydantic request/response schemas (DTOs) with validation rules
    models/            # Auth, logging, rate-limiting, correlation-id, exception-handling
    schemas/           # Environment-based settings (Pydantic Settings), secrets loading
    middlewares/       # Security utilities (JWT, hashing), constants, dependency injection
middleware           # Generic helpers (pagination, formatting, retry decorators)
    config/            # Versioned prompt templates (system, retrieval, citation,
    core/              # Embedding client wrapper, batching, caching logic
wiring               # Vector DB client, index management, hybrid-search implementation
    utils/             # Parsers for PDF/DOCX/TXT, chunking strategies
    prompts/           # Retriever, reranker, context-builder, citation-formatter
summarization)        # StateGraph definitions, node functions, conditional edges, agent
    embeddings/        # Unit, integration, and end-to-end tests (pytest)
    vectorstore/       # Local log output (structured JSON, rotated)
    documents/         # FastAPI application entrypoint / ASGI app factory
    rag/               # Database migration scripts
    langgraph/         # Dockerfile(s), docker-compose for local/dev stacks
flows                # CI/CD pipeline definitions (GitHub Actions)
    tests/             # React/Next.js client application
    logs/              # Chat window, citation cards, upload widget, admin tables
    main.py            # Route-level pages
alembic/              # Data-fetching and auth hooks
docker/               # Documented environment variable template
.github/workflows/    # Python dependency and tooling configuration
frontend/            # Architecture overview and setup instructions
  src/components/
  src/pages/
  src/hooks/
.env.example
pyproject.toml
README.md

```

5.2 Purpose of Each Folder

Folder	Purpose
routers/	Thin HTTP layer; parses requests, calls services, returns schemas. No business logic.
services/	Coordinates use cases (e.g. 'ask a question', 'ingest a document'); the core business logic.
repositories/	Isolates all SQL/ORM access so services never talk to the database directly.
database/	Engine/session setup, connection pooling, and Alembic migration configuration.
models/	Declarative ORM entities mapped to PostgreSQL tables.

Folder	Purpose
schemas/	Pydantic models defining and validating API request/response contracts.
middlewares/	Cross-cutting concerns applied to every request: auth, logging, rate limits.
config/	Centralized, environment-aware configuration (dev/staging/prod).
core/	Security primitives, dependency injection, shared constants.
utils/	Reusable helper functions not tied to a specific domain.
prompts/	Prompt templates kept out of code, versioned, and independently testable.
embeddings/	Wraps the embedding provider; adds batching, retries, and caching.
vectorstore/	Abstracts the vector database so the provider can be swapped with minimal change.
documents/	Format-specific parsing and chunking logic.
rag/	The retrieval-augmented generation pipeline: retrieve, rerank, build context, cite.
langgraph/	Stateful multi-step orchestration graphs and node implementations.
tests/	Automated test suite covering unit, integration, and RAG-quality evaluation.

6. Functional Modules

The platform is decomposed into the following functional modules. Each module is independently testable and maps to a bounded context within the internal company knowledge domain.

Authentication

SSO login (SAML/OIDC) issuing JWTs scoped to the employee's existing identity and group memberships.

User Management

Syncs employee roles and group memberships from the identity provider for permission-aware retrieval.

Admin Dashboard

Connector health, indexing status, and coverage visibility across connected knowledge sources.

Document Upload

Manual upload path plus automated connectors to wikis, drives, and ticketing systems.

Document Processing

Format-aware parsing preserving source permissions metadata alongside content.

Embedding Pipeline

Continuous vectorization of new and updated content across all connected sources.

Vector Database

Permission-tagged vector store enforcing per-user access at retrieval time.

Chat Module

Conversational interface answering natural-language questions with source-linked citations.

Conversation Memory

Session-level context retention for natural follow-up questions.

Search

Standalone hybrid search across all connected sources the user is permitted to view.

Audit Logs

Access-relevant logging supporting security review of sensitive-document queries.

Notifications

Alerts document owners when their content is frequently queried but receives low-confidence answers.

Analytics

Dashboards on top question categories, knowledge gaps, and connector coverage.

Reports

Exportable summaries of documentation gaps for content-owner action.

Settings

Personal notification preferences and default source-scope configuration.

Role Management

RBAC aligned to existing org structure; Admins manage connectors and platform configuration.

7. APIs

7.1 API Design Principles

- RESTful resource-oriented URLs (e.g. /api/v1/documents/{id}).
- Consistent envelope for success and error responses.
- Cursor and offset pagination supported on all list endpoints via ?page & ?page_size.
- Filtering via query parameters (?status=, ?doc_type=, ?date_from=, ?date_to=).
- Sorting via ?sort_by= and ?order=asc|desc.
- Idempotent PUT/DELETE; partial updates via PATCH where applicable.
- All endpoints versioned under /api/v1/ to allow non-breaking evolution.

7.2 Core Endpoints

Method	Endpoint	Description
POST	/api/v1/auth/login	Authenticate user, issue JWT access + refresh tokens.
POST	/api/v1/auth/refresh	Exchange a valid refresh token for a new access token.
GET	/api/v1/users/me	Return the authenticated user's profile and role.
POST	/api/v1/documents	Upload a new source document for ingestion.
GET	/api/v1/documents	List documents with pagination, filtering, and sorting.
GET	/api/v1/documents/{id}	Retrieve a single document's metadata and processing status.
DELETE	/api/v1/documents/{id}	Soft-delete a document and its vector embeddings.
POST	/api/v1/chat/sessions	Create a new conversation session.
POST	/api/v1/chat/sessions/{id}/messages	Submit a question; returns a streamed, cited answer.
GET	/api/v1/chat/sessions/{id}	Retrieve full conversation history.
GET	/api/v1/audit-logs	List audit log entries (Admin only), paginated and filterable.
GET	/api/v1/analytics/usage	Aggregate usage metrics for dashboards.

7.3 Standard Response Envelope

```
{
  "success": true,
  "data": { ... },
  "meta": { "page": 1, "page_size": 20, "total_items": 134, "total_pages": 7 },
  "error": null
}
```

7.4 Standard Error Response

```
{
  "success": false,
  "data": null,
  "meta": null,
  "error": {
    "code": 400,
    "message": "Invalid request"
  }
}
```

```

"error": {
  "code": "DOCUMENT_NOT_FOUND",
  "message": "No document exists with the given ID.",
  "request_id": "3f9a2c11-..."
}
}

```

7.5 Sample Request / Response — Ask a Question

Request:

POST /api/v1/chat/sessions/8f21/messages

Authorization: Bearer <jwt>

Content-Type: application/json

```

{
  "message": "How many days of parental leave am I entitled to, and how do I apply?"
}

```

Response:

```

{
  "success": true,
  "data": {
    "message_id": "9c31-...",
    "answer": "Full-time employees are entitled to 16 weeks of paid parental leave; the application is submitted through the HR portal at least 30 days before the expected leave start date, per the current Parental Leave Policy.",
    "citations": [
      { "source": "HR_Policies/Parental_Leave_Policy_2026.pdf", "page": 4 }
    ],
    "confidence": "high"
  },
  "error": null
}

```

7.6 HTTP Status Codes Used

Code	Meaning	Usage
200	OK	Successful GET/PUT/PATCH.
201	Created	Successful POST creating a resource.
204	No Content	Successful DELETE.
400	Bad Request	Validation failure on request payload.
401	Unauthorized	Missing or invalid JWT.
403	Forbidden	Authenticated but insufficient role/permission.
404	Not Found	Resource does not exist.
409	Conflict	Duplicate resource or state conflict.
422	Unprocessable Entity	Semantically invalid request (e.g. unsupported file type).
429	Too Many Requests	Rate limit exceeded.
500	Internal Server Error	Unhandled server-side exception.

8. Database Design

8.1 Core Tables

Table	Key Columns	Notes
users	id (PK), email, hashed_password, role_id (FK), created_at	One row per authenticated user.
roles	id (PK), name, permissions (JSONB)	Defines RBAC permission sets.
documents	id (PK), title, doc_type, storage_uri, status, uploaded_by (FK users), created_at	Metadata for every ingested source document.
document_chunks	id (PK), document_id (FK), chunk_index, content, token_count, page_number	Chunk-level metadata; vectors live in the vector store, keyed by chunk id.
conversations	id (PK), user_id (FK), title, created_at	One row per chat session.
messages	id (PK), conversation_id (FK), role, content, created_at	Individual turns within a conversation.
citations	id (PK), message_id (FK), document_chunk_id (FK), relevance_score	Links an answer to the exact source chunks used.
audit_logs	id (PK), user_id (FK), action, resource_type, resource_id, ip_address, created_at	Immutable record of security-relevant actions.
notifications	id (PK), user_id (FK), type, payload (JSONB), read_at	In-app notification queue.

8.2 Relationships

- users 1—N documents (uploaded_by)
- users 1—N conversations
- roles 1—N users
- documents 1—N document_chunks
- conversations 1—N messages
- messages 1—N citations
- document_chunks 1—N citations

8.3 Indexing Strategy

- B-tree index on users.email (unique) for fast login lookup.
- Composite index on document_chunks(document_id, chunk_index) for ordered retrieval.
- Index on audit_logs(created_at) to support time-bounded compliance queries.
- Index on messages(conversation_id, created_at) for chronological history retrieval.
- GIN index on documents.metadata (JSONB) for flexible attribute filtering.

8.4 Normalization

The schema is normalized to Third Normal Form (3NF) to eliminate redundant storage of user, document, and conversation attributes. Denormalized read-optimized views are introduced only where analytics dashboards

require aggregate performance, and are refreshed on a schedule rather than stored redundantly in transactional tables.

8.5 Entity-Relationship Summary

```
roles ---< users ---< documents ---< document_chunks ---< citations >--- messages ---<
conversations
users ---< audit_logs
users ---< notifications
```

9. RAG Pipeline

9.1 Document Loader

Format-specific loaders normalize heterogeneous inputs into a common internal representation preserving layout metadata (page, section heading, table boundaries):

Format	Loader Strategy
PDF	Layout-aware extraction (PyMuPDF/pdfplumber) preserving page numbers and table structure; OCR fallback for scanned pages.
DOCX	python-docx traversal preserving headings, tables, and styles for section-aware chunking.
TXT	Encoding-safe text read with paragraph-boundary detection.

9.2 Chunking

Recursive, structure-aware chunking targets 400–800 tokens per chunk with a 10–15% overlap, splitting first on section/heading boundaries, then paragraphs, then sentences, so that no chunk crosses a semantic boundary unnecessarily. Each chunk retains metadata: source document, page, section title, and effective date.

9.3 Embeddings

Chunks and queries are embedded using a bi-encoder sentence-transformer (or an API-based embedding model) into a shared vector space. Embeddings are batched and cached to control latency and cost, and are re-generated whenever the underlying model version changes.

9.4 Vector Database

Vectors are stored with associated metadata in a vector database (pgvector for tight PostgreSQL integration, or a dedicated store such as Qdrant for larger scale), enabling approximate nearest-neighbor search with metadata pre-filtering by document type, role scope, and date range.

9.5 Retriever & Hybrid Search

The retriever combines dense vector similarity with sparse keyword (BM25) search, merging results via reciprocal rank fusion. Hybrid search is critical in this domain because exact terminology, identifiers, and section numbers must be matched precisely, while semantic search captures paraphrased user intent.

9.6 Reranker

A cross-encoder reranker re-scores the top-N hybrid candidates against the exact user query, materially improving precision before the most relevant 4–8 chunks are passed into the LLM's context window.

9.7 Context Injection & Prompt Engineering

Reranked chunks are assembled into a structured context block with explicit source labels. The system prompt instructs the model to answer strictly from the provided context, to state when information is not available, and to attach a citation to every factual claim.

9.8 LLM Response & Citation

The LLM generates an answer grounded in the injected context. The Output Parser cross-checks that every citation in the response maps to an actual retrieved chunk, rejecting or flagging any answer with unverifiable references before it reaches the user.

9.9 Source References

Every answer displays the originating document title, page number, and an excerpt snippet, allowing the user to verify the answer against the primary source in one click — a requirement for any regulated or audit-sensitive workflow.

9.10 Conversation Memory

Short-term memory (the current session's turns) is held in Redis and included in the prompt for coherence. Long-term memory (summarized prior sessions) is persisted in PostgreSQL and selectively retrieved when relevant to the current query.

10. LangChain + LangGraph Flow

10.1 LCEL (LangChain Expression Language)

The pipeline is composed declaratively as a chain of Runnable: prompt | retriever | llm | output_parser. LCEL provides a uniform interface (invoke, batch, stream) across every step, making the pipeline composable, testable in isolation, and easy to reason about.

```
chain = prompt_template | retriever | reranker | llm | output_parser

chain.invoke(query)          # synchronous, single request
chain.batch([q1, q2, q3])    # parallel batch processing
await chain.ainvoke(query)    # async, single request (used in FastAPI routes)
await chain.abatch([...])    # async batch (bulk re-indexing / evaluation jobs)
```

10.2 StateGraph (LangGraph)

For workflows that require more than a single retrieve-then-generate pass, LangGraph models the interaction as a state machine. Each node performs one responsibility (classify intent, retrieve, rerank, generate, validate); edges route the state to the next node based on runtime conditions.

```
START -> classify_intent
classify_intent -> [ retrieve_context | direct_answer | escalate ]    (conditional edge)
retrieve_context -> rerank -> generate_answer
generate_answer -> validate_citations
validate_citations -> [ END | human_approval ]                      (conditional edge)
human_approval -> END
```

10.3 Conditional Edges

Conditional edges inspect the current state (confidence score, document sensitivity flag, citation completeness) and route execution accordingly — for example, sending low-confidence or high-stakes answers to a human-approval node instead of returning them directly to the user.

10.4 Agentic Flow

For multi-step questions (e.g. requiring lookups across multiple document types or a calculation step), LangGraph enables an agent loop where the model can call retrieval or calculation tools iteratively before composing a final answer, with a bounded step count to prevent runaway loops.

10.5 Memory

LangGraph's checkpointing persists the full state of a conversation, enabling pause/resume for human-approval steps and reliable recovery if a node fails mid-execution.

10.6 Fallback Strategy

If the primary LLM provider errors or times out, the graph falls back to a secondary provider or a cached response strategy, and surfaces a clear degraded-mode indicator to the user rather than failing silently.

10.7 Human-in-the-Loop Approval

For high-stakes outputs, LangGraph pauses execution at a `human_approval` node, notifying a reviewer through the Notifications module; the graph resumes only after explicit reviewer sign-off, which is itself recorded in the audit log.

11. Tech Stack

Layer	Technology	Purpose
Frontend	React / Next.js, TypeScript, Tailwind	Chat interface, document management, admin dashboards.
Backend Framework	FastAPI (Python 3.11+)	Async REST API layer.
Orchestration	LangChain (LCEL) + LangGraph	RAG pipeline composition and stateful workflow control.
Primary LLM	OpenAI GPT-4-class models	Answer generation, summarization, structured extraction.
Secondary LLM	Groq-hosted open models	Low-latency fallback and cost-sensitive workloads.
Embedding Model	Sentence-Transformers / OpenAI text-embedding	Dense vector representation of text.
Vector Database	pgvector (or Qdrant for scale)	Semantic similarity search.
Relational Database	PostgreSQL 15+	System of record for users, documents, conversations, audit.
Cache / Session Store	Redis	Caching, rate limiting, session and short-term memory.
Object Storage	S3-compatible blob storage	Durable storage of source documents.
Reverse Proxy	Nginx	TLS termination, load balancing.
Containerization	Docker	Reproducible builds and deployment artifacts.
CI/CD	GitHub Actions	Automated test, build, and deployment pipeline.
Cloud Deployment	AWS / Azure (Kubernetes or ECS)	Managed, autoscaling production hosting.
Observability	OpenTelemetry, Prometheus, Grafana	Metrics, tracing, dashboards, alerting.

12. Production Features

Feature	Implementation Detail
JWT Authentication	Short-lived signed access tokens (15 min) with rotating refresh tokens (7 days), stored httpOnly.
Role-Based Access Control	Declarative permission checks per endpoint, enforced via FastAPI dependencies.
Logging	Structured JSON logs with correlation IDs, shipped to a centralized log store.
Exception Handling	Global exception handlers mapping domain errors to consistent HTTP responses.
Monitoring	OpenTelemetry traces + Prometheus metrics on latency, error rate, and token usage.
Retry Mechanism	Exponential backoff with jitter on LLM/embedding provider calls.
Rate Limiting	Per-user and per-IP token-bucket limits enforced at the gateway and application layer.
Caching	Redis caching of embeddings, frequent queries, and session data.
Pagination	Cursor/offset pagination on every list endpoint with configurable page size.
Search	Hybrid vector + keyword search with metadata filters.
Async APIs	Fully async FastAPI routes and DB access (asyncpg / SQLAlchemy async).
Streaming Response	Server-Sent Events streaming of LLM tokens for perceived low latency.
File Validation	MIME-type, size, and malware-scan checks on every upload.
Security	Encryption in transit (TLS) and at rest, secrets management, input sanitization.
Audit Logs	Immutable, append-only log of every access and administrative action.
Scalability	Stateless API pods behind a load balancer; horizontal autoscaling on CPU/queue depth.

13. Future Enhancements

- Expanded connector library (Slack, Jira, Notion, Zendesk) with incremental sync.
- Proactive knowledge-gap detection surfaced directly to content owners.
- Personalized answer tone/detail level based on role (engineer vs. new hire).
- Slack/Teams native integration for in-context Q&A without leaving the chat tool.
- Automatic flagging of stale documents based on low-confidence answer patterns.

14. Risks

14.1 Technical Risks

Risk	Description	Mitigation
Permission leakage	A retrieved chunk is shown to a user who lacks access to the source document.	Permission metadata enforced at the vector-filter stage, not just at the UI layer, with automated access tests.
Stale index	Connector sync lag causes outdated content to be served as current.	Near-real-time incremental sync with staleness monitoring and alerts.
Answer over-confidence	Assistant states an answer with unwarranted certainty on an ambiguous policy.	Confidence scoring surfaced in the UI; low-confidence answers flagged for verification.
Connector sprawl complexity	Growing number of source systems increases integration and maintenance burden.	Standardized connector framework with shared parsing and permission-mapping logic.

14.2 AI Hallucination Risk

Because some internal questions touch HR, legal, and security policy where incorrect answers can create real employee or organizational risk, an ungrounded or fabricated answer carries real business and compliance risk. The system mitigates this through strict context-grounding (the model is instructed never to answer outside retrieved context), mandatory citation validation, confidence scoring, and human-approval routing for high-stakes responses.

14.3 Mitigation Summary

- Retrieval-grounded generation with explicit 'answer only from context' instructions.
- Citation-integrity validation before any response reaches the user.
- Confidence thresholds that route uncertain answers to human review.
- Continuous evaluation against a labeled question/answer regression set.
- Versioned prompts and models with rollback capability.

15. Development Roadmap

Phase	Weeks	Deliverables
POC	1-3	Single-source connector (e.g. Confluence) ingestion; core RAG chat with citations; internal demo.
MVP	4-8	Multi-source connectors, permission-aware retrieval, RBAC, analytics dashboard, pilot with one department.
Production Hardening	9-11	Security review, load testing, monitoring/alerting, CI/CD pipeline, DR plan.
Deployment	12	Company-wide staged rollout with connector expansion and adoption tracking.

16. Conclusion

Internal knowledge discovery is a universal, high-frequency pain point, and a permission-aware RAG assistant offers a direct, measurable improvement over legacy keyword search: faster answers, less expert interruption, and better visibility into documentation gaps.

Because the productivity gain compounds across every employee and every question asked, this project has a clear, broad-based ROI case that justifies continued investment beyond the POC into a company-wide knowledge platform.

Prepared for approval review — AI Mentor sign-off requested prior to POC development kickoff.